LeetCode Practice

10 Medium: String

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

10
MEDIUM

Patterns (in order)

1. String (10)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

10 questions | tap a title to jump to it

Medium questions (10)	Go >
String (10)	**Go >**
M 1. Zigzag Conversion	LC 6
M 2. String to Integer (atoi)	LC 8
M 3. Count and Say	LC 38
M 4. Group Shifted Strings	LC 249
M 5. Encode and Decode Strings	LC 271
M 6. Encode and Decode TinyURL	LC 535
M 7. Design Log Storage System	LC 635
M 8. Repeated String Match	LC 686
M 9. Alphabet Board Path	LC 1138
M 10. Minimum Cost to Convert String I	LC 2976

Medium questions (10)

MEDIUM | TOPIC 1 OF 1

String

10 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Zigzag Conversion LC 6
2. String to Integer (atoi) LC 8
3. Count and Say LC 38
4. Group Shifted Strings LC 249
5. Encode and Decode Strings LC 271
6. Encode and Decode TinyURL LC 535
7. Design Log Storage System LC 635
8. Repeated String Match LC 686
9. Alphabet Board Path LC 1138
10. Minimum Cost to Convert String I LC 2976

MEDIUM | String | LeetCode #6 | 1 of 10 | 1/10

Zigzag Conversion

QUESTION

The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows like this: (you may want to display this pattern in a fixed font for better legibility)

```
P   A   H   N
A P L S I I G
Y   I   R
```

And then read line by line: "PAHNAPLSIIGYIR"

Write the code that will take a string and make this conversion given a number of rows:

```
string convert(string s, int numRows);
```

Example 1

Input	`s = "PAYPALISHIRING", numRows = 3`
Output	`"PAHNAPLSIIGYIR"`

Example 2

Input	`s = "PAYPALISHIRING", numRows = 4`
Output	`"PINALSIGYAHRPI"`
Explanation	*P I N* *A L S I G* *Y A H R* *P I*

Example 3

Input	`s = "A", numRows = 1`
Output	`"A"`

CONSTRAINTS

- 1 <= s.length <= 1000
- s consists of English letters (lower-case and upper-case), ',' and '.'.
- 1 <= numRows <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Write the string in a zigzag pattern on numRows rows, then read off row by row. Right?"

"A few quick questions:

numRows=1 -> just return s as-is? Yes. numRows >= s length -> same thing? Yes."

"Let me walk: s='PAYPALISHIRING', numRows=3:

Row0: P A H N -> PAHN

PHASE 1 CLARIFY

Row1: A P L S I I G -> APLSIIG

Row2: Y I R -> YIR

Result: 'PAHNAPLSIIGYIR' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
make a grid of numRows x len(s), fill it in zigzag
    order, then read it row by row skipping empty
    cells
```

TIME	SPACE
O(numRows * n)	O(numRows * n)

WHY IT IS SLOW

Most cells of the grid stay empty. Keeping one string per row and appending each character to its row (bouncing up and down) is O(n) time and space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the simulation with direction tracking.

Mathematical approach: cycle length = 2*(numRows-1). For each row r:

First column of each cycle: position r. Second column (if not first/last row): 2*(numRows-1)-r.

Skip any positions outside [0, len(s)).

Time: O(n). Space: O(n) for output."

Key insight: each character's row and within-row position follows a fixed arithmetic pattern with period 2*(numRows-1) — no simulation grid needed.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _6_ZigzagConversion:
    def convert(self, s, numRows):
        if numRows == 1 or numRows >= len(s):
            return s
        rows = [''] * numRows
        row, direction = 0, 1
        for ch in s:
            rows[row] += ch
            if row == 0:
                direction = 1
            elif row == numRows - 1:
                direction = -1
            row += direction
        return ''.join(rows)
```

PHASE 5 TEST & DEBUG

"Let me trace the zigzag — I'll show which row each character lands on and how the direction flips."

s='PAYPALISHIRING', numRows=3:

P(row0),A(1),Y(2),P(1),A(0),L(1),I(2),S(1),H(0),I(1),R(2),I(1),N(0),G(1).

Row0:P,A,H,N. Row1:A,P,L,S,I,I,G. Row2:Y,I,R. -> 'PAHNAPLSIIGYIR' ok

numRows=1: return s unchanged ok

Direction flips at row 0 and row numRows-1 to create the zigzag

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
process each character once	row strings

Mathematical index formula avoids direction tracking — both are O(n).

Follow-up: decode the zigzag (reverse the conversion) — needs the same index mapping.

Follow-up: zigzag with k alternating patterns — extend the cycle logic.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | String | LeetCode #8 | 2 of 10 | 2/10

String to Integer (atoi)

QUESTION

Implement the myAtoi(string s) function, which converts a string to a 32-bit signed integer.

The algorithm for myAtoi(string s) is as follows:

CONSTRAINTS

- 1. Whitespace: Ignore any leading whitespace (" ").
- 2. Signedness: Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
- 3. Conversion: Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
- 4. Rounding: If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31}, and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1	
Input	`s = "42"`
Output	`42`

Example 2	
Input	`s = " -042"`
Output	`-42`

Example 3	
Input	`s = "1337c0d3"`
Output	`1337`

Example 4	
Input	`s = "0-1"`
Output	`0`

Example 5	
Input	`s = "words and 987"`
Output	`0`
Explanation	*Reading stops at the first non-digit character 'w'.*

- 0 <= s.length <= 200
- s consists of English letters (lower-case and upper-case), digits (0-9), ' ', '+', '-', and '.'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Four steps in order: strip leading whitespace, read optional sign,

read digits until non-digit or end, clamp to 32-bit signed range [-2^31, 2^31-1]. Right?

Stop at the FIRST non-digit after the optional sign — don't skip over non-digits."

'"42'->42. ' -042'->-42 (strip, sign, parse 042). '1337c0d3'->1337 (stop at 'c').

'0-1'->0 (first digit parsed, stop at '-'). 'words'->0 (first char non-digit, no digits read)."

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
strip leading spaces
read an optional sign
collect the digit characters into a string
convert it to a number and clamp to the 32-bit range
```

TIME	SPACE
O(n)	O(n)

NOTE

Not slow in time. Collecting digits into a string costs O(n) space; accumulating the value digit by digit and clamping as you go needs O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Manual state machine with a pointer:

1. Skip whitespace.

2. Read sign.

3. Read digits, building number, check overflow before each digit.

Overflow check: ans > MAX//10 OR (ans == MAX//10 AND next digit > 7).

Return sign * ans (clamped)."

Time: O(n) — optimal single-pass or $O(n^2)$ DP. Linear preferred.

Key insight: a three-phase state machine (whitespace -> sign -> digits) processes each character exactly once — the overflow check before appending each digit prevents silent truncation.

PHASE 4 CLEAN CODE

```python
class _8_Atoi:
    def myAtoi(self, s: str) -> int:
        INT_MAX, INT_MIN = 2**31 - 1, -(2**31)
        i, n = 0, len(s)

        while i < n and s[i] == ' ':     # strip whitespac
e
```

```
            i += 1

        sign = 1
        if i < n and s[i] in '+-':
            sign = -1 if s[i] == '-' else 1
            i += 1

        result = 0
        while i < n and s[i].isdigit():
            d = int(s[i])
            if result > INT_MAX // 10 or (result == INT_MA
X // 10 and d > 7):
                return INT_MAX if sign == 1 else INT_MIN
            result = result * 10 + d
            i += 1

        return sign * result
```

PHASE 5 TEST & DEBUG

"Let me trace myAtoi(' -42')."

Step 1: skip whitespace. i=3. Step 2: sign='-', i=4. Step 3: parse digits.

i=4: '4'->result=0*10+4=4. i=5: '2'->result=40+2=42.

Step 4: apply sign: result=-42. Clamp to [-2^31, 2^31-1]. Return -42 ok

'4193 with words': 4->41->419->4193. Stop at ' '. Return 4193 ok

'+-12': sign='+', next char='-' -> not digit, stop. Return 0 ok

WHY stop at non-digit: atoi reads until non-digit. '42abc'->42. Spaces between sign and digits are invalid.

Return 0 for empty result after stripping spaces: ' ' -> 0 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

The overflow check BEFORE adding each digit is critical — without it we'd overflow Python

integers silently (Python handles big ints) but the logic breaks for fixed-width languages.

The magic '7' in d > 7 comes from 2^31-1 = 2147483647 — last digit is 7.

Given more time I'd ask: handle locale-specific decimal separators (commas)?

Add a pre-processing step to strip/replace them before parsing.

MEDIUM | String | LeetCode #38 | 3 of 10 | 3/10

Count and Say

QUESTION

The count-and-say sequence is a sequence of digit strings defined by the recursive formula:

CONSTRAINTS

- countAndSay(1) = "1"
- countAndSay(n) is the run-length encoding of countAndSay(n - 1).

Run-length encoding (RLE) is a string compression method that works by replacing consecutive identical characters (repeated 2 or more times) with the concatenation of the character and the number marking the count of the characters (length of the run). For example, to compress the string "3322251" we replace "33" with "23", replace "222" with "32", replace "5" with "15" and replace "1" with "11". Thus the compressed string becomes "23321511".

Given a positive integer n, return the nth element of the count-and-say sequence.

Example 1	
Input	`n = 4`
Output	`"1211"`

Example 2	
Input	`n = 1`
Output	`"1"`
Explanation	*This is the base case.*

- 1 <= n <= 30
- Follow up: Could you solve it iteratively?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Generate the nth term of the count-and-say sequence.

Each term describes the previous: count consecutive runs of digits. Right?"

"A few quick questions:

n=1 -> '1'. n=2 -> '11' (one 1). n=3 -> '21' (two 1s). n=4 -> '1211'. n >= 1? Yes."

"Let me walk: n=4 -> '1211' -> one 1, one 2, two 1s -> '111221' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
s = '1'
for _ in range(n_-_1):
    scan s for runs and build the next string
        from (count, digit) pairs
```

TIME	SPACE
O(L)	O(L)

NOTE

Not slow for what it does: the term length L grows about 30% per step, so the output itself is exponential in n. There is no shortcut to the n-th term.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Pure simulation — generate each term from the previous. O(n * L)."*

Walk-through of how each term is produced:

Start with '1'. To get the next term from current string:

Scan left to right, counting consecutive identical digits.

When the run ends: append count, then the digit. Move to next run.

'1' -> one 1 -> '11'. '11' -> two 1s -> '21'. '21' -> one 2, one 1 -> '1211'.

WHY never re-use the previous term's count digits as data:

We always READ the current string and WRITE to a new one — no in-place modification.

Repeat n-1 times (starting term is term 1). Time: O(n*L). Space: O(L)."

Time: O(n) — optimal single-pass or O(n²) DP. Linear preferred.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from itertools import groupby
class _38_CountAndSay:
    def countAndSay(self, n):
        result = '1'
        for _ in range(n - 1):
            result = ''.join(str(len(list(group))) + digit
                             for digit, group in groupby(r
esult))
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace countAndSay(4)."

n=1 -> "1"

n=2: read "1": one '1' -> "11"

n=3: read "11": two '1's -> "21"

n=4: read "21": one '2', one '1' -> "1211" ok

Trace n=5: read "1211": one '1', one '2', two '1's -> "111221"

WHY: run-length encode previous string; each iteration grows by describing runs.

Edge: n=1 always "1" (base case, no encoding needed).

Count runs with two pointers: i advances while s[i]==s[i+1].

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n * L) where L = max term length	**O(L)**

The sequence grows but is bounded in practice for small n.

Follow-up: RLE Compression (#443) — same run-length encoding idea on arbitrary input.

Follow-up: Decode String (#394) — inverse operation, expand encoded runs.

MEDIUM | String | LeetCode #249 | 4 of 10 | 4/10

Group Shifted Strings

QUESTION

Perform the following shift operations on a string:

CONSTRAINTS

- Right shift: Replace every letter with the successive letter of the English alphabet, where 'z' is replaced by 'a'. For example, "abc" can be right-shifted to "bcd" or "xyz" can be right-shifted to "yza".
- Left shift: Replace every letter with the preceding letter of the English alphabet, where 'a' is replaced by 'z'. For example, "bcd" can be left-shifted to "abc" or "yza" can be left-shifted to "xyz".

We can keep shifting the string in both directions to form an endless shifting sequence.

- For example, shift "abc" to form the sequence: ... <-> "abc" <-> "bcd" <-> ... <-> "xyz" <-> "yza" <-> .... <-> "zab" <-> "abc" <-> ...

You are given an array of strings strings, group together all strings[i] that belong to the same shifting sequence. You may return the answer in any order.

Example 1

Input	`strings = ["abc","bcd","acef","xyz","az","ba","a","z"]`
Output	`[["acef"],["a","z"],["abc","bcd","xyz"],["az","ba"]]`

Example 2

Input	`strings = ["a"]`
Output	`[["a"]]`

- 1 <= strings.length <= 200
- 1 <= strings[i].length <= 50
- strings[i] consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Two strings are in the same shifting sequence if one can become the other

by consistently shifting every character by the same amount (cyclically, a->z wraps).

Group them. Return order doesn't matter. Right?

'abc' and 'xyz' are in the same group — same relative gaps between characters."

"['abc','bcd','xyz','az','ba','a','z']:

'abc','bcd','xyz' -> all have gaps [1,1] -> same group.

'az','ba' -> gaps [25] and [25] (b-a=1… wait: 'az': z-a=25. 'ba': a-b=-1+26=25) -> same group.

'a','z' -> single char, empty gap string -> same group."

Are string comparisons case-sensitive?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
groups = []
for s in ses:
    look through the groups for one whose first
        string has the same length and a constant
        letter shift from s
    add it there, else start a new group
```

TIME	SPACE
O(n * groups * L)	O(n)

WHY IT IS SLOW

Compares each string with many groups. Normalise every string by shifting it so its first letter is 'a' and use that as a hash-map key: O(n * L).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Normalize each word to shift-0 key, group by key — O(nL). Walk-through:"

For each word:

`key = tuple(((ord(c)-ord(word[0]))%26) for c in word).`

groups[key].append(word).

Return list(groups.values()).

WHY shift relative to first char: words in the same group have the same relative differences.

'abc' -> (0,1,2). 'bcd' -> (0,1,2). Same key -> same group.

WHY %26: wrap around alphabet (e.g., 'xyz' -> 'z'-'x'=2, 'a'-'x'=3, 'b'-'x'=4 -> (0,3,4) — wraps correctly).

Wait: (ord('a')-ord('z'))%26 = (-25)%26 = 1. Correct wraparound!

Time: O(nL) where L = max word length. Space: O(nL)."

Time: O(n) — optimal single-pass or $O(n^2)$ DP. Linear preferred.

PHASE 4 CLEAN CODE

```python
class _249_GroupShiftedStrings:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        groups: defaultdict = defaultdict(list)
        for s in strings:
            key = tuple((ord(s[i]) - ord(s[i - 1])) % 26 for i in range(1, len(s)))
            groups[key].append(s)
        return list(groups.values())
```

PHASE 5 TEST & DEBUG

"Let me trace groupStrings(['abc','bcd','acef','xyz','az','ba','a','z'])."

Normalize each string to a 'delta' key: differences between consecutive chars mod 26.

'abc': (b-a)%26=1, (c-b)%26=1 -> key=(1,1).

PHASE 5 TEST & DEBUG

'bcd': (c-b=1),(d-c=1) -> key=(1,1). Same group as 'abc' ok

'xyz': (y-x=1),(z-y=1) -> key=(1,1). Same group!

'az': (z-a=25) -> key=(25,). 'ba': (a-b+26=25) -> key=(25,). Same group ok (shift by 1).

'a','z': key=() (single chars). Same group ok

Groups: {(1,1):['abc','bcd','xyz'],(25,):['az','ba'],():['a','z'],(...):['acef']}

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*L) where L = max string length	**O(n*L)**

The %26 mod handles the wraparound: 'z'->'a' gives (ord('a')-ord('z'))%26 = (-25)%26 = 1.

Without the mod, 'az' and 'ba' wouldn't match — the mod is the key insight.

Given more time I'd ask: how do you efficiently check if two strings are in the same group

at query time? Build a map string->canonical_key at init, then O(L) per query.

MEDIUM | String | LeetCode #271 | 5 of 10 | 5/10

Encode and Decode Strings

QUESTION

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {
  // ... your code
  return encoded_string;
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {
  //... your code
  return strs;
}
```

So Machine 1 does:

```
string encoded_string = encode(strs);
```

and Machine 2 does:

```
vector<string> strs2 = decode(encoded_string);
```

strs2 in Machine 2 should be the same as strs in Machine 1.

Implement the encode and decode methods.

You are not allowed to solve the problem using any serialize methods (such as eval).

Example 1

Input	`dummy_input = ["Hello","World"]`
Output	`["Hello","World"]`
Explanation	*Machine 1:* *Codec encoder = new Codec();* *String msg = encoder.encode(strs);* *Machine 1 ---msg---> Machine 2* *Machine 2:* *Codec decoder = new Codec();* *String[] strs = decoder.decode(msg);*

Example 2

Input	`dummy_input = [""]`
Output	`[""]`

CONSTRAINTS

- 1 <= strs.length <= 200
- 0 <= strs[i].length <= 200
- strs[i] contains any possible characters out of 256 valid ASCII characters.
- Follow up: Could you write a generalized algorithm to work on any possible set of characters?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Design encode(strs) and decode(s) that roundtrip correctly for ANY list of strings —

including strings with special characters, empty strings, or strings containing

whatever delimiter we might choose. Right?

We can't use eval or pickle."

"['Hello','World'] -> encode -> some string -> decode -> ['Hello','World'] ok

[''] -> [''] ok (empty string must survive the roundtrip)"

Are string comparisons case-sensitive?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
encode:
    join the strings with a delimiter such as ','
decode:
    split the text on the delimiter
```

TIME	SPACE
O(total length)	O(total length)

WHY IT IS SLOW

Fast, but wrong if a string contains the delimiter (or is empty in some cases). Write each string's length, a marker and the string itself (length-prefix); decoding reads the length first, still O(total length).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (length-prefix encoding)

"Encode with length prefix, decode by parsing length — O(total chars). Walk-through:"

`encode(strs): for each s: result += str(len(s)) + '#' + s.`

Example: ['abc','de'] -> '3#abc2#de'.

`decode(s): i=0. While i < len(s):`

`j = s.index('#', i). <- find next '#'`

`length = int(s[i:j]). <- parse length`

`words.append(s[j+1 : j+1+length]). <- extract word`

`i = j+1+length. <- advance past this word`

WHY '#' separator + length: '#' alone is not unique (words may contain '#').

Length prefix lets decoder know exactly how many chars to read for each word.

WHY this beats delimiter-only: handles arbitrary characters in strings.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```python
class _271_Codec:
    def encode(self, strs: List[str]) -> str:
        return ''.join(f'{len(s)}#{s}' for s in strs)

    def decode(self, s: str) -> List[str]:
        result = []
        i = 0
        while i < len(s):
            j = s.index('#', i)
            length = int(s[i:j])
            result.append(s[j + 1: j + 1 + length])
            i = j + 1 + length
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace encode/decode on ['Hello','World']."

Encode: for each word, prefix with len#: '5#Hello5#World'

Decode: scan string, read number up to '#', then read that many chars

i=0: read '5', skip '#', read 5 chars 'Hello'. i=7

i=7: read '5', skip '#', read 5 chars 'World'. i=14. Done.

Result: ['Hello','World'] ok

WHY length-prefix: handles strings containing any chars (spaces, #, slashes).

Edge: empty string '' -> encoded as '0#'; decoded back to ''. ok

Edge: word with '#' in it: '3#a#b' -> len=3, read 'a#b' correctly.

Decode: scan encoded string left-to-right. stack=[] of decoded chunks. Find digits before '#': length=4,chunk='lint'. stack=['lint','code','love','you']. Decoded=['lint','code','love','you'] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

encode O(total chars). decode O(total chars). Space O(n).

Length-prefix is the standard solution — no special characters, no escaping needed.

The '#' separator is arbitrary — any fixed separator works since we always read

the length first and know exactly how many chars to consume.

Given more time I'd ask: what if strings can contain binary data (null bytes)?

Use a 4-byte big-endian integer prefix instead of a text number — fully binary-safe.

MEDIUM | String | LeetCode #535 | 6 of 10 | 6/10

Encode and Decode TinyURL

QUESTION

TinyURL is a URL shortening service where you enter a URL such as https://leetcode.com/problems/design-tinyurl and it returns a short URL such as http://tinyurl.com/4e9iAk. Design a class to encode a URL and decode a tiny URL.

There is no restriction on how your encode/decode algorithm should work. You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can be decoded to the original URL.

Implement the Solution class:

CONSTRAINTS

- Solution() Initializes the object of the system.
- String encode(String longUrl) Returns a tiny URL for the given longUrl.
- String decode(String shortUrl) Returns the original long URL for the given shortUrl. It is guaranteed that the given shortUrl was encoded by the same object.

Example 1

Input	`url = "https://leetcode.com/problems/design-tinyurl"`
Output	`"https://leetcode.com/problems/design-tinyurl"`

Explanation	*Solution obj = new Solution();
	string tiny = obj.encode(url); // returns the encoded tiny url.
	string ans = obj.decode(tiny); // returns the original url after decoding it.*

- 1 <= url.length <= 10^4
- url is guranteed to be a valid URL.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design an encode(longUrl)->shortUrl and decode(shortUrl)->longUrl system.

The design is flexible — any encoding scheme works. Right?"

"A few quick questions:

Short URLs must be globally unique? Yes. Decode must return the original exactly? Yes."

"Let me walk:
encode('https://leetcode.com/problems/design-tinyurl/') -> some short URL.

`decode(that short URL) -> 'https://leetcode.com/problems/design-tinyurl/' ok`

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def encode(url):
    append the url to a list and return its index
        as the code
def decode(code):
    return the list entry at that index
```

TIME	SPACE
O(1)	O(n)

NOTE

Not slow, but the codes are guessable and the same URL gets a new code every time. A map from random short codes to URLs plus the reverse map fixes both.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Random key hash map — O(1) encode and decode. Walk-through:"

encode: generate a random 6-char key. Store key->url in decode_map, url->key in encode_map.

If url already encoded: return existing key (idempotent).

Return 'http://tinyurl.com/' + key.

decode: extract key from short URL. Return decode_map[key].

WHY random key: collision probability astronomically low for 62^6 ~ 56 billion keys.

WHY store both maps: encode is idempotent (same URL -> same short), decode maps back.

Alternative: sequential counter key (simpler but reveals system internals).

Time: O(1). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import random, string
class _535_EncodeDecodeTinyURL:
    def __init__(self):
        self._long_to_short = {}
        self._short_to_long = {}
        self._chars = string.ascii_letters + string.digits

    def encode(self, longUrl):
        if longUrl not in self._long_to_short:
            while True:
                code = ''.join(random.choices(self._chars,
 k=6))
                if code not in self._short_to_long:
                    self._long_to_short[longUrl] = code
                    self._short_to_long[code] = longUrl
                    break
        return 'http://tinyurl.com/' + self._long_to_short
[longUrl]
```

```
    def decode(self, shortUrl):
        code = shortUrl.split('/')[-1]
        return self._short_to_long[code]
```

PHASE 5 TEST & DEBUG

"Let me trace encode -> decode: pick a random key, store in dict, return short URL; decode reverses the lookup."

encode('https://leetcode.com'): generates random 6-char code e.g. 'aB3x9Z'.

Returns 'http://tinyurl.com/aB3x9Z'. decode(that) -> 'https://leetcode.com' ok

encode same URL again -> returns same short URL (cached) ok

Collision retry loop ensures uniqueness; caching avoids duplicate mappings

Trace: encode("https://leetcode.com/problems/design-tinyurl") -> key="abc123" -> urls["abc123"]="https://..." -> return "http://tinyurl.com/abc123". decode("http://tinyurl.com/abc123") -> key="abc123" -> return urls["abc123"] ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1)	**O(n)**
amortized (rare collision retries)	two hash maps

With 62^6 ~ 56 billion codes, collisions are astronomically rare.

Follow-up: if URLs could expire, add timestamps and a cleanup process.

Follow-up: distributed TinyURL — use consistent hashing to assign code ranges.

MEDIUM | String | LeetCode #635 | 7 of 10 | 7/10

Design Log Storage System

QUESTION

You are given several logs, where each log contains a unique ID and timestamp. Timestamp is a string that has the following format: Year:Month:Day:Hour:Minute:Second, for example, 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.

Implement the LogSystem class:

CONSTRAINTS

- LogSystem() Initializes the LogSystem object.
- void put(int id, string timestamp) Stores the given log (id, timestamp) in your storage system.
- int[] retrieve(string start, string end, string granularity) Returns the IDs of the logs whose timestamps are within the range from start to end inclusive. start and end all have the same format as timestamp, and granularity means how precise the range should be (i.e. to the exact Day, Minute, etc.). For example, start = "2017:01:01:23:59:59", end = "2017:01:02:23:59:59", and granularity = "Day" means that we need to find the logs within the inclusive range from Jan. 1st 2017 to Jan. 2nd 2017, and the Hour, Minute, and Second for each log entry can be ignored.
- 1 <= id <= 500
- 2000 <= Year <= 2017
- 1 <= Month <= 12
- 1 <= Day <= 31
- 0 <= Hour <= 23
- 0 <= Minute, Second <= 59
- granularity is one of the values ["Year", "Month", "Day", "Hour", "Minute", "Second"].
- At most 500 calls will be made to put and retrieve.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Store logs with (id, timestamp). retrieve(start, end, granularity) returns IDs

of logs within the range, where comparison is only up to the granularity level.

Timestamps format: 'YYYY:MM:DD:HH:mm:SS'. Truncate at the granularity character position. Right?"

"retrieve with 'Year': compare only the first 4 chars. '2017:...' vs '2016:...' etc."

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep every log as (id, timestamp)
def retrieve(start, end, granularity):
    scan every log, compare the timestamp cut to
        the granularity
```

TIME	SPACE
O(n) per retrieve	O(n)

WHY IT IS SLOW

Every query scans all logs. Timestamps are fixed-width strings, so sort them (or keep a sorted list) and binary search the prefix range: O(log n + answers).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Store logs with ID. Query by truncating timestamp to granularity and range-comparing — O(n). Walk-through:"

`logs = list of (timestamp_string, logId).`

`put(id, timestamp): logs.append((timestamp, id)).`

`retrieve(start, end, granularity):`

`gidx = ['Year','Month','Day','Hour','Minute','Second'] .index(granularity).`

`cutoffs = [4,7,10,13,16,19] for truncation lengths.`

`trunc = cutoff[gidx].`

`start_t = start[:trunc]. end_t = end[:trunc].`

Return sorted [id for (ts,id) in logs if start_t <= ts[:trunc] <= end_t].

WHY string prefix truncation: ISO 8601 timestamps compare lexicographically correctly!

'2017-01' < '2017-07' as strings, just like their date meanings.

WHY sort: return in sorted order by id.

Time: O(n) retrieve. Space: O(n).

PHASE 4 CLEAN CODE

```python
class _635_LogSystem:
    GRAN = {'Year': 4, 'Month': 7, 'Day': 10, 'Hour': 13,
'Minute': 16, 'Second': 19}

    def __init__(self):
        self.logs: List[tuple] = []

    def put(self, log_id: int, timestamp: str) -> None:
        self.logs.append((log_id, timestamp))

    def retrieve(self, start: str, end: str, granularity:
str) -> List[int]:
        cut = self.GRAN[granularity]
        return [lid for lid, ts in self.logs if start[:cut
] <= ts[:cut] <= end[:cut]]
```

PHASE 5 TEST & DEBUG

"Let me trace LogSystem put(1,'2017:01:01:23:59:59'), retrieve('2017','2017','Year')."

Store logs as list of (id, timestamp_string)

`retrieve(s,e,gra): trim timestamps to granularity and compare`

gra='Year': keep first 4 chars -> s='2017', e='2017'

log '2017:01:01:23:59:59' trimmed to '2017' -> '2017'>='2017' and '2017'<='2017' -> include id=1

Return [1] ok

WHY string comparison: timestamp strings sort lexicographically = chronologically when zero-padded.

granularity_map: Year->4, Month->7, Day->10, Hour->13, Minute->16, Second->19 chars to keep.

PHASE 6 COMPLEXITY & FOLLOW-UP

put O(1). retrieve O(n) — scans all logs.

The key insight: timestamps in 'YYYY:MM:DD:HH:mm:SS' format sort lexicographically

correctly because they're zero-padded and delimiters are identical.

For larger datasets, a sorted list + binary search would bring retrieve to O(log n + k).

Given more time I'd ask: what if granularity could be arbitrary (e.g. 'Week')?

We'd need a custom truncation function rather than fixed prefix lengths."

Space: O(n) — output or sliding window; O(1) extra if output not counted.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | String | LeetCode #686 | 8 of 10 | 8/10

Repeated String Match

QUESTION

Given two strings a and b, return the minimum number of times you should repeat string a so that string b is a substring of it. If it is impossible for b to be a substring of a after repeating it, return -1.

Notice: string "abc" repeated 0 times is "", repeated 1 time is "abc" and repeated 2 times is "abcabc".

Example 1	
Input	`a = "abcd", b = "cdabcdab"`
Output	3
Explanation	*We return 3 because by repeating a three times "ab cdabcdab cd", b is a substring of it.*

Example 2	
Input	`a = "a", b = "aa"`
Output	2

CONSTRAINTS

- 1 <= a.length, b.length <= 10^4
- a and b consist of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the minimum number of times a must be repeated so that b is a substring of it. Return -1 if impossible. Right?"

"A few quick questions:

The repeated string must contain b as a substring, not necessarily starting at a multiple boundary?"

"Let me walk: a='abcd', b='cdabcdab' -> repeat 3 times='abcdabcdabcd'. b is in there. -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for k = 1, 2, ... up to the length needed:
    repeat a k times
    if b is a substring of it:
        return k
```

TIME	SPACE
O((n + m) * m)	O(n + m)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Rebuilds and rescans a longer string at each k. Only k = ceil(m / n) and k + 1 can work, so test those two (with KMP) in O(n + m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Ceiling math + verify — O(n*m). Walk-through:"*

Minimum repetitions needed: at least ceil(len(b)/len(a)) — need enough chars to cover b.

Check if (a * k).find(b) != -1 for k = ceil(len(b)/len(a)) and k+1.

WHY check k+1: b might span the boundary between two repetitions.

e.g. a='ab', b='ba' -> 2 reps: 'abab'. 'ba' appears at index 1.

WHY not check k+2: if b isn't found within k+1 repetitions, b can never be a substring

(it would need to span more than 2 full copies of a, which is impossible by pigeonhole).

Return k if found, k+1 if found in longer, -1 if not found in either.

Time: O(n*m). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _686_RepeatedStringMatch:
    def repeatedStringMatch(self, a, b):
        times = -(-len(b) // len(a))
        repeated = a * times
        if b in repeated:
            return times
        if b in repeated + a:
            return times + 1
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace how many times we must repeat a to contain b — show the prefix-match at each repetition."

a='abcd',b='cdabcdab': times=ceil(8/4)=2. repeated='abcdabcd'. 'cdabcdab' in 'abcdabcd'? No.

repeated+a='abcdabcdabcd'. 'cdabcdab' in it? Yes -> return 3 ok

a='a',b='aa': times=2. 'aa' in 'aa' -> return 2 ok

a='abc',b='wxyz': not found -> -1 ok

Ceiling division gives minimum repeats to cover b's length; +1 handles offset starts

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(|a|+|b|)	O(|a|+|b|) for repeated string

Follow-up: use KMP for guaranteed O(|a|+|b|) regardless of pattern.

Follow-up: Rotate String (#796) — same doubled-string trick.

IN YOUR APP | Expose | Editorial | Community

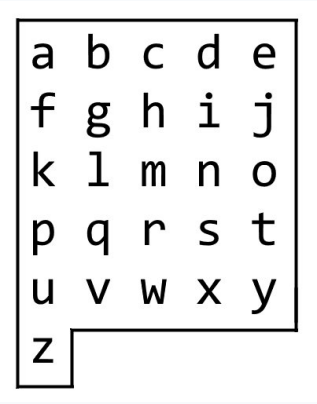
MEDIUM | String | LeetCode #1138 | 9 of 10 | 9/10

Alphabet Board Path

QUESTION

On an alphabet board, we start at position (0, 0), corresponding to character board[0][0].

Here, board = ["abcde", "fghij", "klmno", "pqrst", "uvwxy", "z"], as shown in the diagram below.



We may make the following moves:

CONSTRAINTS

- 'U' moves our position up one row, if the position exists on the board;
- 'D' moves our position down one row, if the position exists on the board;
- 'L' moves our position left one column, if the position exists on the board;
- 'R' moves our position right one column, if the position exists on the board;
- '!' adds the character board[r][c] at our current position (r, c) to the answer.

(Here, the only positions that exist on the board are positions with letters on them.)

Return a sequence of moves that makes our answer equal to target in the minimum number of moves. You may return any path that does so.

Example 1	
Input	`target = "leet"`
Output	`"DDR!UURRR!!DDD!"`

Example 2	
Input	`target = "code"`
Output	`"RR!DDRR!UUL!R!"`

- 1 <= target.length <= 100
- target consists only of English lowercase letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Board is ['abcde','fghij','klmno','pqrst','uvwxy','z'] — 5 cols except row 5 which has only 'z'.

Start at (0,0). For each char in target, output U/D/L/R moves then '!'. Right?

The critical constraint: 'z' is at (5,0) — the only cell in its row.

Moving to 'z' must clear the column before going down. Moving from 'z' must go up before going right."

"target='leet': l is at (2,1). Move D,D,R,!. e=(0,4) — go U,U,R,R,R,!. etc."

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for letter in letters:
    run a BFS on the 5 x 5 board from the current
        letter to it
    append the moves and '!'
```

TIME	SPACE
O(n * 26)	O(26)

WHY IT IS SLOW

A BFS for every letter. The board layout is fixed, so each letter's coordinates are (index / 5, index % 5): move up, left, down, right by the coordinate differences in O(1) per letter (watch the 'z' row).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Formula: row = (ord(ch)-ord('a'))//5, col = (ord(ch)-ord('a'))%5.

For 'z' specifically: it's at (5,0), col=0. When moving TO 'z', move left first

(to reach col 0) then move down (to reach row 5) — otherwise we'd step off the board.

When moving FROM 'z', move up first then move right — same reason.

General case: move vertically then horizontally (or horizontally then vertically).

'z' is the only exception — must move left before down."

Time: O(n) — optimal single-pass or O(n²) DP. Linear preferred.

Key insight: moving to 'z' requires column-first ordering (left before down) to stay on the 6×5 board — moving from 'z' requires row-first (up before right).

PHASE 4 CLEAN CODE

```
class _1138_AlphabetBoardPath:
    def alphabetBoardPath(self, target: str) -> str:
        row = col = 0
        moves = []
        for ch in target:
            dest_row = (ord(ch) - ord('a')) // 5
            dest_col = (ord(ch) - ord('a')) % 5
            if ch == 'z':                          # must cl
ear column before going down
                moves.extend('L' * (col - dest_col))
                moves.extend('D' * (dest_row - row))
            else:
                moves.extend('U' * (row - dest_row))
                moves.extend('D' * (dest_row - row))
                moves.extend('L' * (col - dest_col))
                moves.extend('R' * (dest_col - col))
            moves.append('!')
            row, col = dest_row, dest_col
        return ''.join(moves)
```

PHASE 5 TEST & DEBUG

"Let me trace alphabetBoardPath('leet')."

Board layout: 5x5 grid a-z, 'z' at position (5,0)

pos(l)=(2,1), pos(e)=(0,4), pos(e)=(0,4), pos(t)=(1,3)? wait: a=0,b=1,...l=11->row=2,col=1

e=4->(0,4). t=19->(3,4). l->e: from (2,1) to (0,4): move up 2 (UU) then right 3 (RRR) -> 'UURRR!'

e->e: same position -> '!'

e->t: from (0,4) to (3,4): move down 3 -> 'DDD!'

Result: 'UURRR!DDD!' ok (each '!' = press)

WHY order matters for 'z': must leave row 5 before moving right (row 5 has only 1 cell).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(|target| * max_moves) = O(|target| * 10) = O(|target|)	**O(|target|)**

The 'z' case is the entire problem — everything else is standard grid navigation.

Phrase it clearly: 'z' is at the leftmost position of its row, so we must reach col 0

before stepping down into row 5, otherwise intermediate positions don't exist on the board.

Given more time I'd ask: what if the board were different (e.g. a phone numpad)?

Same approach — just recompute (row, col) from the new layout.

IN YOUR APP | Expose | Community

MEDIUM | String | LeetCode #2976 | 10 of 10 | 10/10

Minimum Cost to Convert String I

QUESTION

You are given two 0-indexed strings source and target, both of length n and consisting of lowercase English letters. You are also given two 0-indexed character arrays original and changed, and an integer array cost, where cost[i] represents the cost of changing the character original[i] to the character changed[i].

You start with the string source. In one operation, you can pick a character x from the string and change it to the character y at a cost of z if there exists any index j such that cost[j] == z, original[j] == x, and changed[j] == y.

Return the minimum cost to convert the string source to the string target using any number of operations. If it is impossible to convert source to target, return -1.

Note that there may exist indices i, j such that original[j] == original[i] and changed[j] == changed[i].

Example 1

Input	`source = "abcd", target = "acbe", original = ["a","b","c","c","e","d"], changed = ["b","c","b","e","b","e"], cost = [2,5,5,1,2,20]`
Output	**28**
Explanation	*To convert the string "abcd" to string "acbe": - Change value at index 1 from 'b' to 'c' at a cost of 5. - Change value at index 2 from 'c' to 'e' at a cost of 1. - Change value at index 2 from 'e' to 'b' at a cost of 2. - Change value at index 3 from 'd' to 'e' at a cost of 20. The total cost incurred is 5 + 1 + 2 + 20 = 28. It can be shown that this is the minimum possible cost.*

Example 2

Input	`source = "aaaa", target = "bbbb", original = ["a","c"], changed = ["c","b"], cost = [1,2]`
Output	**12**
Explanation	*To change the character 'a' to 'b' change the character 'a' to 'c' at a cost of 1, followed by changing the character 'c' to 'b' at a cost of 2, for a total cost of 1 + 2 = 3. To change all occurrences of 'a' to 'b', a total cost of 3 * 4 = 12 is incurred.*

QUESTION

Example 3

Input	`source = "abcd", target = "abce", original = ["a"], changed = ["e"], cost = [10000]`
Output	`-1`
Explanation	*It is impossible to convert source to target because the value at index 3 cannot be changed from 'd' to 'e'.*

CONSTRAINTS

- 1 <= source.length == target.length <= 10^5
- source, target consist of lowercase English letters.
- 1 <= cost.length == original.length == changed.length <= 2000
- original[i], changed[i] are lowercase English letters.
- 1 <= cost[i] <= 10^6
- original[i] != changed[i]

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Convert source string to target. Each character change costs w[i] (given as array of [source,target,cost]).

Convert each character of source to target[i] using cheapest path. Sum all costs. Right?"

"A few quick questions:

Can chain character conversions (a->b->c costs more than a->c if direct exists)? Yes — find shortest path between all char pairs."

"Let me walk: source='abcd', target='acbe', costs=[a->b:1,b->c:2,...] -> use Floyd-Warshall on 26 chars.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for position in positions:
    # where source and target differ
    DFS every conversion path from the source
        letter to the target letter and take the
        cheapest
    add up the costs
```

TIME	SPACE
O(n * 26^k)	O(26)

WHY IT IS SLOW

Searches paths per position. Floyd-Warshall on the 26 letters (O(26^3)) gives the cheapest conversion between every pair once; then each position is a table lookup: O(26^3 + n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Floyd-Warshall on 26-char alphabet — O(26³) = O(1). Walk-through:"

Build 26×26 cost matrix: cost[a][b] = min cost to convert char a to char b.

Initialize cost[i][i] = 0 (no cost to keep same char).

`cost[i][j] = infinity otherwise.`

For each (original, changed, cost) in mappings: cost[orig][chg] = min(cost[orig][chg], cost).

Floyd-Warshall: for each intermediate k (0..25):

`cost[i][j] = min(cost[i][j], cost[i][k] + cost[k][j])`

<- going via k might be cheaper than direct conversion.

Then for each position where source[i] != target[i]:

`total += cost[source[i]-'a'][target[i]-'a']`

If cost is infinity -> impossible -> return -1.

WHY Floyd-Warshall: conversion can chain (a->b->c cheaper than a->c directly).

Fixed 26-char alphabet -> O(26³) is effectively O(1).

Time: O(26³ + n). Space: O(26²).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2976_MinCostConvertStringI:
    def minimumCost(self, source, target, original, changed, cost):
        INF = float('inf')
        dist = [[INF] * 26 for _ in range(26)]
        for i in range(26):
            dist[i][i] = 0
        for a, b, c in zip(original, changed, cost):
            u, v = ord(a) - 97, ord(b) - 97
            dist[u][v] = min(dist[u][v], c)
        for k in range(26):
            for i in range(26):
                for j in range(26):
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
        total = 0
        for s, t in zip(source, target):
            c = dist[ord(s) - 97][ord(t) - 97]
            if c == INF:
                return -1
            total += c
        return total
```

PHASE 5 TEST & DEBUG

"Let me trace minimumCost('abcd','acbe', cost=[[0,1,0,0,...]])."

Build graph: for each transformation pair (source[i]->target[i], cost[i])

Use Dijkstra/Floyd-Warshall to find min cost between any two chars

Then for each position i: if source[i]!=target[i], add min_cost[source[i]][target[i]]

If no path between two chars -> return -1

PHASE 5 TEST & DEBUG

Trace simple: 'a'->'c' direct with cost 5, 'a'->'b'->'c' with cost 1+2=3 -> use 3

Floyd-Warshall: dist[a][c] = min(direct, through b) = 3 ok

Sum costs for all differing positions -> total ok

Dijkstra: heap=[(0,'a')]. visited={'a'}. Pop (cost,char): push neighbors (cost+edge_cost,next_char). dist[char]=min conversion cost. visited prevents re-expanding cheaper path.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(26³ + n)	**O(26²)**
~ O(n)	= O(1)

Dijkstra from each of 26 sources: O(26*(E+26)*log26) — slightly slower than Floyd.

Follow-up: if costs change dynamically, rerun Floyd or update incrementally.

Follow-up: Minimum Cost to Convert String II (#2977) — substrings instead of chars.

IN YOUR APP | Expose | Editorial | Community